Experimental Results

PCCST503 Machine Learning — Assignment 1 — Safe Semantic Planner

1. Method

Environment. Intel Xeon @ 2.10 GHz, single core, Linux 6.18 x86-64, g++ 13.3.0, compiled with `-O2 -std=c++17 -Wall -Wextra -Wpedantic` (no warnings). Timing uses `std::chrono::steady_clock`. All runs are deterministic: the random graphs come from a fixed-seed linear congruential generator, so every number below reproduces exactly.

Metrics. The eight quantities requested by the brief are collected as follows.

Metric	How it is measured
Goal success rate	`PlanningResult::success` over all runs
Bad states visited	states on the returned path with membership in B
Total path cost	sum of *nominal* transition costs, not the weighted metric
Minimum distance to bad states	min over path states of clear(s)
Number of explored states	vertex expansions in `computeShortestPath`
Planning time	search only; setup reported separately
Memory usage	sum of `capacity()` over all planner-owned containers
Replanning time	search time of the `plan()` call following a change

Correctness oracle. Every test compares $g(s_I)$ against `DijkstraPlanner`, a separate implementation of the same `Planner` interface that uses no heuristic. Agreement is reported per test as PASS or FAIL.

Reproduce. `./safe_planner --tests`, `--bench`, `--sweep`, `--scaling`, `--demo`.

2. Test cases

2.1 Summary

Test	Scenario	Path returned	Cost	Min clear.	Bad	Expanded	Time (ms)	Optimal
1	Basic reachability	S→A→B→G	3.000	n/a	0	4	0.0012	PASS
2a	Bad state X present	S→C→D→G	3.600	2.000	0	4	0.0003	PASS

Test	Scenario	Path returned	Cost	Min clear.	Bad	Expanded	Time (ms)	Optimal
2b	X cleared at runtime	S→A→X→G	3.000	n/a	0	3	0.0003	PASS
2c	C becomes bad	S→A→X→G	3.000	1.414	0	0	0.0001	PASS
3a	Safety margin, $\lambda = 0$	S→P1→P2→G	3.000	1.077	0	4	0.0005	PASS
3b	Safety margin, $\lambda = 6$	S→Q1→Q2→G	4.800	3.000	0	5	0.0004	PASS
4a	Initial plan	S→A→G	3.000	n/a	0	4	0.0006	—
4b	Agent at A, (A,G) blocked	A→D→G	2.200	n/a	0	2	0.0004	PASS
5a	Goal G1	S→A→G1	2.000	n/a	0	3	0.0002	—
5b	Goal moved to G2	S→A→B→G2	3.000	n/a	0	4	0.0002	PASS
6a	Before insertion	S→A→B→G	3.000	n/a	0	4	0.0002	—
6b	Shortcut (S,B) added	S→B→G	1.500	n/a	0	1	0.0001	PASS

Goal success rate: 12/12 (100%). Bad states visited: 0 in every run. Optimality: PASS on every check.

2.2 Test 1 — basic reachability

The unique valid path is returned. Effective cost 3.030 exceeds nominal cost 3.000 because the reliability term $\mu(1 - r)$ charges for the imperfect reliabilities 0.99, 0.98, 0.97. Four expansions for a four-state graph confirms each state is expanded once.

2.3 Test 2 — bad state avoidance, including runtime changes

With X bad, the planner rejects the cheaper 3.000 route and returns the 3.600 route — the requested behaviour. Score rises from 99.97 to 119.31 despite the higher cost, because the clearance term $\gamma D = 10 \times 2.0$ dominates the extra 0.6 of cost.

Clearing X at runtime returns the planner to the cheap route in 3 expansions. Making C bad instead requires **0 expansions**: the current path (S→A→X→G) does not touch C and remains optimal, so the propagation reaches no vertex that changes the answer. Detecting "nothing to do" for free is exactly what incremental replanning should buy.

2.4 Test 3 — safety margin

Two routes, identical endpoints. With $\lambda = 0$ the planner minimises cost alone and takes the short route at clearance 1.077. With $\lambda = 6, d_{safe} = 3.5$ it pays 60% more cost (3.000 → 4.800) to raise clearance 2.8× (1.077 → 3.000). Reported score rises from 110.74 to 128.17.

The full trade-off curve is in §4.

2.5 Test 4 — dynamic transition, with agent motion

The initial plan is S→A→G at cost 3.000. The agent then *executes* the first step and only afterwards discovers that (A, G) is unavailable — the realistic ordering, and the case D* Lite exists for. Repair from A takes **2 expansions** and returns A→D→G. The start move itself costs nothing: k_m absorbs the heuristic shift and no stored value is discarded.

2.6 Test 5 — goal update

Moving the goal from G1 to G2 yields S→A→B→G2 in 4 expansions. As §6.3 of the design report explains, g and rhs must be reset because they are goal-relative, but the adjacency lists, embeddings, clearance field and heap storage are all reused: the reset is $O(n)$ with zero reallocation.

2.7 Test 6 — transition addition

Inserting the shortcut (S, B) at cost 0.5 halves the solution, 3.000 → 1.500, in **one expansion**. The insertion also lowers η from 1.000 to 0.500 (the new edge covers 2 units of distance for 0.5 of cost), which triggers the open-list re-key described in §6.4 of the design report. Without that re-key the search can return a stale path here.

3. Benchmark: 150 × 150 lattice

22,500 states, 178,808 transitions, 120 bad states, 8-connected, corner to corner.

Configuration	Expanded	Search time	Cost	Min clear.	Memory
D* Lite, Euclidean heuristic	6,169	4.06 ms	222.36	2.828	3.05 MB
Same planner, $h = 0$	22,380	13.54 ms	222.36	2.828	3.05 MB

Heuristic contribution: 3.63× fewer expansions, 3.33× less time, with identical cost. The ablation forces $\eta = 0$ on the same planner with the same heap and the same weights, so the only variable is the heuristic.

Path length 156 states, 0 bad states visited, minimum clearance 2.828 against $d_{safe} = 3.0$.

3.1 Incremental repair after 300 random removals

	Expanded	Search time	Effective cost
Incremental repair	1,259	0.76 ms	223.825
Replanned from scratch	6,146	3.71 ms	223.825

4.88× fewer expansions, 4.89× faster, identical cost. Equal cost across two independent searches is evidence the repair is exact, not approximate.

4. Parameter study: what λ actually controls

Three routes between the same endpoints, $d_{safe} = 3.5$, λ swept $0 \to 4$.

λ	Route	Cost	Clearance	Score(P)
0.00 – 1.50	S→P1→P2→G	3.000	1.077	110.74
1.75 – 4.00	S→Q1→Q2→G	4.800	3.000	128.17

Closed-form check. The low route has effective cost $3.000 + 1.527\lambda$, the high route $4.800 + 0.336\lambda$. They cross at $\lambda^* = 1.511$. The experiment switches between $\lambda = 1.50$ and $\lambda = 1.75$, matching the prediction and confirming the weight formula is implemented as specified.

A negative result worth reporting. The graph also contains a middle route (cost 3.900, clearance 1.887). It is **never selected at any** λ. It is not Pareto-dominated — nothing in the graph is both cheaper and safer — but in the (cost, penalty) plane it lies *above* the segment joining the other two options, and minimising a linear combination can only ever return a vertex of the lower convex hull.

This is a real limitation of the objective form suggested in the brief, not of the search: $\text{Score}(P) = \alpha G - \beta C + \gamma D + \delta R$ is linear, and linear scalarisation cannot reach non-convex portions of a Pareto front. Anyone tuning λ hoping for a moderate compromise route will fail without ever being told why. Obtaining such plans requires a different combination rule: a hard constraint $\text{clear}(s) \geq d_{min}$ on every visited state, a lexicographic objective, or maximising minimum clearance subject to a cost budget.

5. Scaling study

8-connected lattices, corner to corner, $|B| = n/200$, $\lambda = 3.0$, $d_{safe} = 3.0$.

5.1 Initial plan

N	States	Transitions	Setup (ms)	Search (ms)	Expanded	Memory (KB)
40	1,600	12,324	0.40	0.26	418	226
80	6,400	50,244	1.86	0.91	1,465	893
120	14,400	113,764	4.95	3.24	4,883	1,995
160	25,600	202,884	11.27	5.42	7,570	3,549
200	40,000	317,604	23.48	8.84	11,734	5,532

Search time grows slightly faster than linearly in n (25× states → 34× time), consistent with the $\log n$ factor from the heap. Memory is linear in $n + m$ as predicted, at roughly 142 bytes per state including both adjacency directions.

Setup dominates at large n — 23.5 ms against 8.8 ms of search at $N = 200$ — and it is almost entirely the $O(nkd)$ clearance field: 40,000 states × 200 bad states = 8M distance evaluations. This is the single clearest optimisation target, and a k-d tree would reduce it to $O(n \log k)$.

5.2 Incremental repair vs. replanning from scratch

Two change regimes per problem size. "Localised" removes 200 random transitions. "Severe" removes every third transition on the current solution path.

N	Change	Repair (ms)	Repair exp.	Scratch (ms)	Scratch exp.	Speed-up
40	200 random	0.06	107	0.24	409	**3.80×**
40	1/3 of path cut	0.28	366	0.22	386	0.80×
80	200 random	0.07	133	0.93	1,480	**12.95×**
80	1/3 of path cut	1.52	2,217	1.05	1,661	0.69×
120	200 random	0.73	1,191	3.27	4,884	**4.51×**
120	1/3 of path cut	2.92	4,110	3.20	4,782	1.09×
160	200 random	0.71	1,016	5.73	7,556	**8.07×**
160	1/3 of path cut	8.10	10,884	6.82	9,540	0.84×
200	200 random	0.14	182	9.43	11,726	**67.80×**
200	1/3 of path cut	20.87	25,577	10.68	14,037	0.51×

Two clear regimes:

Localised changes: 3.8× to 67.8× faster. The speed-up is not a function of problem size but of how much of the search tree the change invalidates. At $N = 200$, 200 removals scattered over 317,604 transitions mostly miss the solution corridor entirely, so repair touches 182 vertices against 11,726 for a fresh search.

Severe changes: 0.51× to 1.09×, i.e. usually a loss. Cutting a third of the solution path forces D* Lite through its under-consistent branch: affected g values must first be driven to infinity and propagated, and the region must then be re-expanded. The vertex can be processed twice, so repair does up to 1.8× the work of a fresh search.

This is a property of the algorithm, not a defect in the implementation, and the practical consequence is concrete: a deployed planner should track the fraction of the tree a change invalidates and fall back to a fresh search past a threshold. The crossover in this data sits near a repair/fresh expansion ratio of 1.0, which for these lattices corresponds to roughly 15–20% of the solution path being cut.

6. Cross-implementation validation

The same design was implemented independently in C11 (`safe_planner.c`, 1,077 lines) and C++17 (`safe_planner.cpp`, 1,496 lines), sharing no code. Using the same LCG seed for the benchmark graph,

the two produce **bit-identical** paths, nominal costs, effective costs, clearances, scores, expansion counts and optimality verdicts across all twelve test-case runs and the benchmark.

Two independent implementations agreeing exactly is stronger evidence of correctness than either one passing its own tests, and it is the check that caught the missing open-list re-key of design report §6.4 during development.

7. Summary against the brief's evaluation criteria

Criterion	Result
Goal success rate	100% (12/12 test-case runs, plus benchmark and all scaling runs)
Bad states visited	**0** everywhere — enforced structurally, not by penalty
Total path cost	Provably optimal under the weighted metric; verified against Dijkstra on every run
Minimum distance to bad states	Tunable via λ; 2.8× improvement demonstrated, crossover matches closed form
Number of explored states	3.63× reduction from the heuristic; 4.88× further from incremental repair
Planning time	4.06 ms for 22,500 states / 178,808 transitions; 8.84 ms at 40,000 states
Memory usage	Linear in $n + m$; 5.4 MB of planner state at 40,000 states
Replanning time	0.76 ms vs 3.71 ms from scratch for localised changes; measured honestly in both regimes